

Vibrational Deduction Transformers: Laplacian Wave-Based Recurrent Transformer

Lorenzo Moriondo

Independent Researcher — tuned.org.uk

ORCID: 0000-0002-8804-2963

2 June 2026

Abstract

We develop a self-contained theory of *vibrational learning* on feature space and use it to construct the *Vibrational Deduction Transformer* (VDT), a forward-only recurrent architecture whose intermediate states are constrained by the spectral geometry of a feature-space graph Laplacian. The foundations are classical and mechanical. The transpose $A^\top$ of a data matrix transports residuals from sample space to feature space; equipping that feature space with a graph Laplacian L_f built from $A^\top$ as a stiffness operator and a positive diagonal mass matrix M as inertia turns it into a mass–spring system in the sense of Rayleigh. The pair (L_f, M) defines a generalised eigenproblem $L_f u = \lambda M u$ whose eigenvalues are squared natural frequencies and whose eigenvectors are mode shapes, and the generalised Rayleigh quotient $R_M(z) = z^\top L_f z / z^\top M z$ measures the spectral energy of any direction. From this footing we recover Laplacian-preconditioned gradient descent with a provable linear-convergence guarantee, taumode frequency regularisation that couples learning to the energy-dispersion structure of a corpus, and a second-order damped-oscillator reading of gradient descent itself. We then lift these mechanics from a single parameter vector to the hidden states of a recurrent transformer. Each forward pass alternates an expressive transformer mixing block with a structured *vibrational state update* — a discrete damped wave operator Φ_L on the feature Laplacian — so that every recurrent step is forced through a physically interpretable spectral manifold, in direct analogy to the lattice-projection step of the Lattice Deduction Transformer [15]. We further equip the state with a *signed density matrix* $\varrho_t = \varrho_t^+ - \varrho_t^-$ (each component positive semidefinite) in the Laplacian eigenbasis, capturing positive and negative evidential mass, mode–mode coherence, and interferometric cancellation. We give the architecture, its spectral (modal) decoupling, the training objectives, and an evaluation protocol mirroring the Lattice Deduction Transformer benchmark exactly, with interpretable modal energy spectra as a diagnostic.

1 Introduction

Reasoning in large neural networks has become largely synonymous with the *chain-of-thought* paradigm [2] and its test-time-compute extensions [3, 17], in which a model produces explicit intermediate tokens before committing to a final answer. The appeal is transparency: each reasoning step is a readable string, and generating more tokens at inference time can improve accuracy [18]. Yet the paradigm couples *computation* to *communication*: every intermediate thought must be expressed as a sequence of vocabulary items, and the cost of additional reasoning is proportional to the number of output tokens rather than to the intrinsic difficulty of the computation being performed.

An alternative — closer in spirit to classical automated reasoning — is to treat inference as *iterative refinement of a structured latent state*, analogous to discrete diffusion over a partial-information representation [4, 15]. Here, no intermediate tokens are emitted; instead the network repeatedly updates a hidden state constrained to lie in a domain with algebraic or geometric semantics. Each update is a provably structured step — a partial deduction in a logical lattice [15], a diffusion step on a discrete space [4] — and the number of iterations trades off compute against solution quality without touching the output vocabulary.

This paper proposes a third, *physically grounded* instance of the same paradigm. We equip the latent state with the spectral geometry of a *feature-space graph Laplacian* L_f , built from the transposed data matrix $A^\top$, and constrain each recurrent step to propagate along the eigenmodes of (L_f, M) via a discrete damped wave equation. The Laplacian provides a *discrete geometry* — a spectral lattice of natural frequencies — and the vibrational operator provides the *forward approximation mechanism*, in direct analogy to the role played by lattice projection in the Lattice Deduction Transformer [15]. Where chain-of-thought externalises reasoning into tokens, and where lattice-deduction internalises it into logical candidate sets, the *Vibrational Deduction Transformer* (VDT) internalises it into the *modal dynamics of feature space*: a physical wave settles toward an answer the way a struck string settles toward silence, each recurrent step dissipating energy in spectrally structured fashion.

Two ideas motivate this paper, one about *learning dynamics* and one about *architecture*, and our central claim is that they are the same idea applied at two scales.

The first idea is that supervised learning on a linear model is already a mechanical process. Given a data matrix $A \in \mathbb{R}^{m \times n}$ and a target b , the gradient of the least-squares loss is $\frac{1}{m}A^\top(Ax - b)$, and the transpose $A^\top$ is precisely the operator that carries the residual from *sample space* back into *feature space*, where the parameters live. If we additionally endow feature space with a graph Laplacian L_f built from $A^\top$ and a mass matrix M , the resulting pair (L_f, M) is a mass-spring system in the sense of Rayleigh’s *Theory of Sound* [1]: its generalised eigenvalues are squared natural frequencies, its eigenvectors are mode shapes, and gradient descent in this geometry behaves like a damped oscillator settling in a potential well. This vibrational reading yields a mass-aware Laplacian preconditioner with provable linear convergence and a spectral (taumode) regulariser that biases learning toward the energy-dispersion bands of a corpus.

The second idea concerns *structured intermediate states* in recurrent transformers. Transformers excel at flexible computation but impose no constraint on intermediate representations: every hidden state may occupy any point in $\mathbb{R}^d$, and structure must be learned entirely from data. Recent work argues that forcing representations through a constrained domain at each recurrent step is a strong inductive bias for iterative reasoning. Lattice Deduction Transformers [15] make this concrete for logical deduction: every recurrent step projects its latent state onto a lattice of candidate sets, so inference becomes a sequence of provably sound partial derivations.

We unify the two. The mass-spring system of the first idea *is* the constrained domain of the second. The eigenmodes of L_f are the “pure tones” of the feature graph; their mixtures are spectral chords; and a second-order damped wave step on this modal lattice is a structural operator that constrains recurrent hidden states in exact analogy to lattice projection in LDT. Where LDT projects onto a logical lattice, the VDT propagates along a *spectral* lattice governed by a physical wave equation.

Contributions.

- **Foundations (Sections 2–7).** A self-contained mass-weighted vibrational interpretation of feature-space Laplacians: the dual-space role of $A^\top$, the pair (L_f, M) and the generalised Rayleigh quotient R_M , a mass-aware Laplacian preconditioner $S_{\sigma, M}$ with a linear convergence guarantee, a taumode frequency regulariser, and the damped-oscillator reading of gradient descent that underlies the wave update.

- **Architecture (Section 8).** A *vibrational deduction transformer* that replaces lattice projection with Laplacian modal propagation — a second-order discrete wave update Φ_L with damping and forcing controlled by attention context — as the structured intermediate operator in a recurrent transformer.
- **Signed density state (Section 9).** A signed density matrix $\varrho_t = \varrho_t^+ - \varrho_t^-$ in the Laplacian eigenbasis, supporting positive/negative evidential mass and interferometric cancellation, with measurable modal observables.
- **Objectives and protocol (Sections 10–11).** A depth-supervised training objective combining task loss, taumode frequency regularisation, and an occupancy penalty, and an evaluation protocol designed for exact comparability with the LDT benchmark, using modal energy spectra as interpretable diagnostics.

Organisation. Section 2 recalls the dual-space role of the transpose. Section 3 constructs the mass-weighted feature graph and the generalised Rayleigh quotient. Section 4 defines the Laplacian-regularised cost, Section 5 the preconditioned gradient method and its convergence, Section 6 the taumode regulariser, and Section 7 the vibrational (second-order) reading of gradient descent. Sections 8–11 build the VDT on these foundations. We close with discussion, limitations, and conclusions.

Part I — Foundations: Vibrational Learning on Feature Space

2 Dual spaces and the role of the transpose

Let $V \cong \mathbb{R}^m$ be the sample space and $W \cong \mathbb{R}^n$ the feature space. We interpret the data matrix $A \in \mathbb{R}^{m \times n}$ as a linear map

$$A : W \longrightarrow V, \quad x \mapsto Ax. \quad (1)$$

Under the standard Euclidean inner products on V and W , the transpose $A^\top$ is the adjoint map $A^\top : V \rightarrow W$, satisfying

$$\langle Ax, v \rangle_V = \langle x, A^\top v \rangle_W \quad \text{for all } x \in W, v \in V. \quad (2)$$

Thus $A^\top$ transports vectors in sample space into feature space, and can be viewed as a concrete realisation of the passage to the dual space V^* .

Given a parameter vector $x \in W$, the residual is $e(x) = Ax - b \in V$, the least-squares cost is $J(x) = \frac{1}{2m} \langle e(x), e(x) \rangle_V$, and its gradient is

$$\nabla J(x) = \frac{1}{m} A^\top e(x) = \frac{1}{m} A^\top (Ax - b). \quad (3)$$

The factor $A^\top$ plays two roles. *Dimensional alignment:* the residual lives in $V \cong \mathbb{R}^m$ whereas the parameter vector lives in $W \cong \mathbb{R}^n$, and $A^\top$ maps the residual into W , producing a valid direction in parameter space. *Error attribution:* each row of $A^\top$ corresponds to a feature, and $A^\top e(x)$ computes the correlation between each feature and the residual, attributing portions of the error to coordinates of x . The normal equations $A^\top (Ax^* - b) = 0$ then require that every feature functional in this dual space sees no correlation with the residual. When A is rank-deficient or non-square the least-squares solution may be expressed via the Moore–Penrose pseudoinverse $A^+ = \lim_{\delta \rightarrow 0} (A^\top A + \delta I)^{-1} A^\top$; see [5].

3 Feature graph and mass-weighted Rayleigh quotient

We endow the feature space W with additional structure by constructing a mass-weighted graph whose nodes are features and whose edges encode feature–feature similarity. Crucially, this graph is built from the transposed data $A^\top$.

Let $X = A^\top \in \mathbb{R}^{n \times m}$. The i -th row $X_{i,:} \in \mathbb{R}^m$ is the signal of feature i across all samples. We construct an undirected weighted graph $G_f = (V_f, E_f, W_f)$ with node set $V_f = \{1, \dots, n\}$ and symmetric weight matrix W_f with entries

$$(W_f)_{ij} = \kappa(X_{i,:}, X_{j,:}), \quad 1 \leq i, j \leq n, \quad (4)$$

where κ is a similarity kernel, for example a Gaussian kernel $\kappa(u, v) = \exp(-\|u - v\|_2^2 / \sigma^2)$, a cosine similarity, or a correlation measure. In practice W_f is sparsified by keeping each node’s k nearest neighbours. This data-driven construction of a Laplacian from observed signals follows work on learning graph Laplacians for smooth graph-signal representations [11], the key difference being that the “signals” are feature profiles across samples rather than item-level observations.

3.1 Laplacian, mass matrix, and Rayleigh quotient

The degree matrix is $(D_f)_{ii} = \sum_j (W_f)_{ij}$ and the combinatorial graph Laplacian is $L_f = D_f - W_f$. It is symmetric positive semidefinite, and for any $z \in \mathbb{R}^n$,

$$z^\top L_f z = \frac{1}{2} \sum_{i,j} (W_f)_{ij} (z_i - z_j)^2, \quad (5)$$

which measures the smoothness of z on the graph: it vanishes if and only if z is constant on each connected component [6, 7].

Remark 3.1 (Combinatorial vs. normalised Laplacian). We use the combinatorial Laplacian $L_f = D_f - W_f$ throughout. For heterogeneous degrees the symmetric normalised Laplacian $\mathcal{L} = D_f^{-1/2} L_f D_f^{-1/2}$ is often preferable: its spectrum lies in $[0, 2]$ regardless of graph scale, making R_M more comparable across graphs. With $M = D_f$, the generalised eigenproblem $L_f u = \lambda D_f u$ is equivalent to the $\mathcal{L}$ eigenproblem, so the two formulations coincide under the degree-mass choice.

To obtain a vibrational system in the sense of Rayleigh we introduce a positive definite diagonal mass matrix $M \in \mathbb{R}^{n \times n}$, with $M_{ii} > 0$ assigning a mass to each feature.

Canonical constructions of M .

- *Degree mass* (default): $M = D_f$, free as a by-product of building L_f . This turns $L_f u = \lambda M u$ into the symmetric normalised Laplacian eigenproblem; highly connected features receive higher mass, matching the intuition that heavier oscillators are harder to move.
- *ArrowSpace taumode mass*: when an ArrowSpace index is available, set $M_{ii} = (1 - \lambda_i^\tau + \varepsilon)^{-1}$, where $\lambda_i^\tau \in [0, 1]$ is the taumode score of feature i . Spectrally rough, high Rayleigh-energy features receive higher mass, coupling inertia to the ArrowSpace spectral geometry.

The pair (L_f, M) defines a generalised eigenproblem

$$L_f u_k = \lambda_k M u_k, \quad (6)$$

whose eigenvalues $\lambda_k \geq 0$ are interpreted as squared natural frequencies of the feature-space mass–spring system, and whose eigenvectors u_k are mode shapes. For any nonzero $z \in \mathbb{R}^n$ the generalised Rayleigh quotient is

$$R_M(z) = \frac{z^\top L_f z}{z^\top M z}, \quad (7)$$

playing exactly the role of Rayleigh’s quotient in vibration analysis: if z aligns with an eigenmode u_k then $R_M(z) = \lambda_k$, and more generally $R_M(z)$ is an energy-weighted average of eigenvalues [12]. A natural Markov transition matrix on the feature graph is

$$P_f = D_f^{-1}W_f, \quad (8)$$

whose i -th row holds the transition probabilities of a random walk on features. Thus L_f and M encode the geometry and inertia of feature space induced by $A^\top$, while P_f encodes the pathways along which information flows between features. These three objects — L_f , M , and P_f — recur throughout the architecture of Part II.

3.2 Relationship to ArrowSpace and Graph Wiring

The feature Laplacian L_f and the taumode coordinates above are exactly the constructions used by the ArrowSpace/Graph-Wiring spectral search framework [13, 14]. There, a feature Laplacian built from a corpus matrix $A^\top$ is paired with taumode values $\lambda_i^r \in [0, 1)$ that compress the Rayleigh energy of each corpus vector into a one-dimensional spectral coordinate for vector search and retrieval. In this paper the same L_f , its eigendecomposition $L_f = U\Lambda U^\top$, the mass matrix M , and the prototype taumode values all serve a learning role: they define the spectral lattice on which the vibrational dynamics of Part II propagate.

4 A mass-weighted Laplacian-regularised cost

Equipped with L_f and M , we define a Laplacian-regularised least-squares functional

$$J_\lambda(x) = \frac{1}{2m}\|Ax - b\|_2^2 + \frac{\lambda}{2}x^\top L_f x, \quad x \in \mathbb{R}^n, \quad (9)$$

with $\lambda \geq 0$. The first term enforces data fidelity; the second penalises lack of smoothness of x across the feature graph, so that strongly connected features are encouraged to have similar coefficients, mirroring classical graph-based semi-supervised learning [8, 9, 10]. Using (7), the smoothness term factors as

$$x^\top L_f x = R_M(x) x^\top M x, \quad (10)$$

an interpretive lens reading the penalty as a mass-weighted squared frequency $R_M(x)$ times a mass-weighted energy $x^\top M x$.

The gradient is $\nabla J_\lambda(x) = \frac{1}{m}A^\top(Ax - b) + \lambda L_f x$, and the stationary condition is

$$\left(\frac{1}{m}A^\top A + \lambda L_f\right)x = \frac{1}{m}A^\top b, \quad (11)$$

so the Laplacian regulariser modifies the classical normal equations by adding a graph-based smoothness term in feature space.

5 Laplacian-preconditioned gradient descent

We now describe a learning algorithm that uses the mass-weighted feature-space Laplacian to define the pathways along which gradients are diffused at each step.

5.1 Explicit and implicit smoothing

For the unregularised cost $J(x) = \frac{1}{2m} \|Ax - b\|_2^2$, gradient descent updates $x_{t+1} = x_t - \eta \frac{1}{m} A^\top (Ax_t - b)$, using $A^\top$ purely to move the residual into feature space. To incorporate feature-graph pathways we smooth the gradient $g_t = \nabla J(x_t)$. The *explicit* smoothing is

$$\tilde{g}_t = (I - \sigma L_f) g_t, \quad (12)$$

which is positive semidefinite only when $\sigma < 1/\lambda_{\max}(L_f)$; violating this bound makes the operator indefinite and can diverge. The *implicit* mass-aware smoothing uses the resolvent

$$S_{\sigma,M} = (M + \sigma L_f)^{-1} M, \quad \tilde{g}_t = S_{\sigma,M} g_t, \quad (13)$$

which is positive semidefinite for *all* $\sigma \geq 0$ and $M \succ 0$, removing the stability restriction on σ — the main reason it is preferred. $S_{\sigma,M}$ is one step of implicit Euler discretisation of the mass-weighted heat equation $M\dot{z} + L_f z = 0$ on the feature graph; setting $M = I$ recovers the isotropic smoothing $(I + \sigma L_f)^{-1}$ [12]. The implicit operator $S_{\sigma,M}$ reappears in Part II as the stable alternative to the explicit wave step.

5.2 Convergence in the quadratic case

Writing $H = \frac{1}{m} A^\top A$ and $P_{\sigma,M} = S_{\sigma,M}$, the preconditioned update is $x_{t+1} = x_t - \eta P_{\sigma,M} H(x_t - x^*)$, where x^* is the unique minimiser of J whenever $H \succ 0$.

Proposition 5.1 (Linear convergence under mass-aware smoothing). *Assume $H \succ 0$ and $L_f \succeq 0$ symmetric, with $M \succ 0$ diagonal. For any $\sigma > 0$ the preconditioner $P_{\sigma,M} = (M + \sigma L_f)^{-1} M$ is symmetric positive definite, and the spectrum of $\mathcal{H}_{\sigma,M} = P_{\sigma,M} H$ lies in $(0, \infty)$. Let $\mu_{\sigma,M} = \lambda_{\min}(\mathcal{H}_{\sigma,M})$ and $L_{\sigma,M} = \lambda_{\max}(\mathcal{H}_{\sigma,M})$. If $0 < \eta < 2/L_{\sigma,M}$, then in the $P_{\sigma,M}^{-1}$ -weighted norm $\|w\|_{P^{-1}}^2 = w^\top P_{\sigma,M}^{-1} w$,*

$$\|x_{t+1} - x^*\|_{P^{-1}} \leq (1 - \eta \mu_{\sigma,M}) \|x_t - x^*\|_{P^{-1}},$$

and since $P_{\sigma,M} \succ 0$ makes this norm equivalent to the Euclidean norm, the iterates converge linearly to x^ .*

Proof sketch. (i) Since $M \succ 0$ is diagonal and $L_f \succeq 0$, $A_\sigma = M + \sigma L_f \succ 0$. Writing $A_\sigma = M^{1/2}(I + \sigma M^{-1/2} L_f M^{-1/2}) M^{1/2}$ shows $P_{\sigma,M} = A_\sigma^{-1} M$ is congruent via $M^{1/2}$ to the SPD matrix $(I + \sigma M^{-1/2} L_f M^{-1/2})^{-1}$, so $P_{\sigma,M} \succ 0$. (ii) Because $P_{\sigma,M} \succ 0$ and $H \succ 0$, $\mathcal{H}_{\sigma,M}$ shares its spectrum with the SPD matrix $P_{\sigma,M}^{1/2} H P_{\sigma,M}^{1/2}$, hence all positive. (iii) The error obeys $e_{t+1} = (I - \eta \mathcal{H}_{\sigma,M}) e_t$; under $\tilde{e} = P_{\sigma,M}^{-1/2} e$ the iteration matrix becomes the symmetric $I - \eta P_{\sigma,M}^{1/2} H P_{\sigma,M}^{1/2}$, whose eigenvalues lie in $[\mu_{\sigma,M}, L_{\sigma,M}] \subset (0, \infty)$. For $0 < \eta < 2/L_{\sigma,M}$ they all lie in $(-1, 1)$, giving the stated contraction; norm equivalence transfers it to $\|\cdot\|_2$. The full argument is in Appendix A. $\square$

The spectrum of $\mathcal{H}_{\sigma,M}$ is fully characterised by the eigenvalues of $S_{\sigma,M} H = (M + \sigma L_f)^{-1} M (A^\top A/m)$, so the allowable step size and contraction factor are computable from the preconditioned Hessian: Laplacian smoothing changes the geometry of the level sets through $S_{\sigma,M}$, but the standard spectral theory of gradient descent on quadratics continues to apply. Proposition 5.1 concerns the base convex model and is independent of the non-convex taumode regulariser of Section 6.

5.3 Algorithmic summary

We summarise mass-aware Laplacian-preconditioned gradient descent as Algorithm 1.

Mass-Weighted Laplacian-Preconditioned Gradient Descent

Input: $A \in \mathbb{R}^{m \times n}$, target $b \in \mathbb{R}^m$, step size $\eta > 0$, diffusion $\sigma \geq 0$, iterations T .

Preprocessing:

1. Form $X = A^\top$. Build affinity W_f from rows of X via a kernel κ and sparsification.
2. Compute D_f , $L_f = D_f - W_f$, and a positive diagonal mass matrix M (degree mass or ArrowSpace taumode mass).
3. Optionally form $S_{\sigma,M} = (M + \sigma L_f)^{-1}M$.

For $t = 0, \dots, T - 1$:

1. Residual $r_t = Ax_t - b$; raw gradient $g_t = \frac{1}{m}A^\top r_t$.
2. Smooth: explicit $\tilde{g}_t = (I - \sigma L_f)g_t$ or implicit $\tilde{g}_t = S_{\sigma,M}g_t$.
3. Update $x_{t+1} = x_t - \eta \tilde{g}_t$.

Output: x_T .

Algorithm 1: Mass-weighted Laplacian-preconditioned gradient descent.

6 Taumode frequency regularisation

This section is a speculative extension: the taumode regulariser J_{freq} is a research direction rather than a validated component, and Proposition 5.1 is independent of it. The mass-spring system (L_f, M) lets us go beyond smoothness penalties and bias the frequencies of update directions toward the spectral bands occupied by in-distribution corpus items in ArrowSpace.

At iteration t consider trial shapes $q_j(x_t) \in \mathbb{R}^n$, e.g. $q_1 = \nabla J(x_t)$ and optionally Krylov directions $q_2 = L_f \nabla J(x_t)$. Rather than computing an exact Rayleigh quotient per trial shape at every step, we use precomputed taumode values as a low-dimensional spectral proxy. Let $\{\phi_c\}_{c=1}^C$ be feature-space prototypes (Graph Wiring centroids) with taumode values $\lambda_c^\tau \in [0, 1)$ computed offline by ArrowSpace [13, 14]. Projecting $q_j(x_t)$ onto the prototypes with softmax similarity weights $w_{j,c}(x_t)$ yields a differentiable approximate frequency

$$\hat{\rho}_j(x_t) = \sum_{c=1}^C w_{j,c}(x_t) \lambda_c^\tau, \quad (14)$$

a one-dimensional summary of the spectral position of $q_j(x_t)$ relative to the corpus taumode spectrum. From the empirical taumode distribution we estimate a fundamental scale $\lambda_\star^\tau$ as a robust central statistic (the median of the 25–75th percentile band), clamping $\hat{\rho}_j$ to this band so the regulariser acts chiefly when frequencies leave the in-distribution range. The taumode frequency MSE term and full functional are

$$J_{\text{freq}}(x) = \frac{1}{2J} \sum_{j=1}^J (\hat{\rho}_j(x) - \lambda_\star^\tau)^2, \quad J_{\lambda,\mu}(x) = \frac{1}{2m} \|Ax - b\|_2^2 + \frac{\lambda}{2} x^\top L_f x + \mu J_{\text{freq}}(x). \quad (15)$$

Since $\hat{\rho}_j$ depends nonlinearly on x , J_{freq} is generally non-convex and is best viewed as a heuristic spectral bias; the base quadratic-with-preconditioning model remains convex. This taumode MSE reappears verbatim, lifted to the recurrent trajectory, as the frequency loss $\mathcal{L}_{\text{freq}}$ of the VDT in Section 10.

7 Gradient descent as a settling vibrational system

The preconditioned method above is first-order. The deeper mechanical content — and the bridge to the architecture of Part II — comes from treating learning as a *second-order* damped vibration.

7.1 Loss landscapes as vibrational potentials

Treat the parameters $x \in \mathbb{R}^n$ as generalised coordinates of a fictitious mechanical system and the loss $J(x)$ as its potential energy. With kinetic energy $\frac{1}{2}\dot{x}^\top M \dot{x}$ and a damping matrix $C \succeq 0$ we consider the damped second-order dynamics

$$M\ddot{x}(t) + C\dot{x}(t) + \nabla J(x(t)) = 0, \quad (16)$$

the analogue of Rayleigh’s damped vibration with ∇J as the restoring force. Near a non-degenerate minimum $x^\star$ with Hessian $H = \nabla^2 J(x^\star) \succ 0$, writing $u = x - x^\star$,

$$M\ddot{u} + C\dot{u} + Hu = 0, \quad (17)$$

exactly the form analysed by Rayleigh: M is mass, H is stiffness, and the vibrational modes are governed by the eigenpairs of $M^{-1}H$, with squared frequencies given by the generalised Rayleigh quotient $R_M(z) = z^\top H z / z^\top M z$. Low $R_M(z)$ are slow modes of the optimiser; high values are stiff directions.

7.2 From overdamped descent to explicit vibration

Standard gradient descent $x_{k+1} = x_k - \eta \nabla J(x_k)$ is the explicit Euler discretisation of the first-order flow $\dot{x} = -\nabla J(x)$, recovered from (16) in the overdamped limit $M \rightarrow 0$, $C = I$. Thus ordinary descent explores only the zero-inertia regime. Retaining inertia and discretising the full second-order system with step h (and $M = I$ for simplicity),

$$v_{k+1} = v_k - h \nabla J(x_k) - \gamma h v_k, \quad (18)$$

$$x_{k+1} = x_k + h v_{k+1}, \quad (19)$$

a discrete analogue of $\ddot{x} + \gamma \dot{x} + \nabla J(x) = 0$. Along each eigenvector z of H with $H z = \lambda z$, the mode obeys $\ddot{u}^\lambda + \gamma \dot{u}^\lambda + \lambda u^\lambda = 0$ — a damped harmonic oscillator with natural frequency $\sqrt{\lambda}$. Rayleigh’s classification applies directly: underdamped ($\gamma^2 < 4\lambda$, oscillatory decay), critical ($\gamma^2 = 4\lambda$, fastest non-oscillatory return), and overdamped ($\gamma^2 > 4\lambda$, slow monotone relaxation). Choosing M to encode feature masses and C for mode-dependent damping yields a spectrum-aware optimiser interpretable through Rayleigh’s analysis.

Remark 7.1 (The seed of the wave update). Equations (18)–(19) are the scalar, single-parameter prototype of the matrix-valued vibrational state update Φ_L that drives the VDT (Section 8.4). There the stiffness H is replaced by the feature Laplacian L_f , the single coordinate x by a full token–feature state Q_t , and the constant forcing by an attention-derived term B_t . The damped-oscillator modal decoupling of this section is precisely what makes the architecture’s spectral formulation possible.

Part II — Architecture: The Vibrational Deduction Transformer

8 The Vibrational Deduction Transformer

We now lift the mechanics of Part I from a single parameter vector to the hidden states of a recurrent transformer. The mass–spring system (L_f, M) becomes the constrained domain through which every recurrent hidden state must pass, playing the role that the logical lattice plays in the Lattice Deduction Transformer [15].

8.1 Lattice Deduction Transformers as structural inspiration

Davis et al. [15] introduce the LDT, a recurrent transformer that approximates logically sound deduction by repeatedly projecting its hidden state onto a lattice of candidate hypotheses. Each recurrent step comprises a standard transformer layer and a lattice projection $\Pi_{\mathcal{L}}$ mapping the latent state to the nearest admissible lattice element. The structured intermediate state acts as a powerful inductive bias: by constraining what the network can “think” at each step, the architecture converges faster and generalises better on logical and arithmetic reasoning. We replace $\Pi_{\mathcal{L}}$ with the Laplacian modal operator Φ_L , and replace the logic lattice with the spectral lattice of L_f from Part I.

8.2 Overall recurrent structure

Let $X_0 \in \mathbb{R}^{n \times d}$ be the input token features after embedding, with n tokens and feature dimension d . The VDT iterates a depth- K recurrence

$$X_{t+1} = \text{TransformerBlock}(X_t + \Phi_L(X_t, Q_{t-1}, Q_t)), \quad t = 1, \dots, K, \quad (20)$$

where Φ_L is the vibrational state block (Section 8.4) and $Q_t \in \mathbb{R}^{n \times d}$ is the vibrational state, an explicit second-order memory. We initialise $Q_0 = Q_1 = X_0$ and update Q_{t+1} via the discrete wave equation at each step. The recurrent depth K is a hyperparameter; all transformer blocks share weights across recurrent steps, following the Universal Transformer convention [16] adopted also by LDT.

8.3 Feature graph construction

Given $X_0 \in \mathbb{R}^{n \times d}$ we construct a feature graph G_f over the d feature coordinates, exactly as in Section 3 but now per-batch and online. Let $\tilde{X} = X_0^\top \in \mathbb{R}^{d \times n}$, whose i -th row is the signal of feature i across all n tokens. The affinity is the Gaussian kernel of Section 3,

$$(W_f)_{ij} = \exp(-\|\tilde{X}_{i:} - \tilde{X}_{j:}\|_2^2 / \sigma_f^2), \quad i \neq j; \quad (W_f)_{ii} = 0, \quad (21)$$

sparsified by top- k neighbours, with D_f , $L_f = D_f - W_f$, and $P_f = D_f^{-1}W_f$ defined as before. For a sequence task, G_f can be fixed once per batch or updated per recurrent step at modest cost.

8.4 Vibrational state block

The vibrational state block implements a discrete second-order wave equation on the feature graph — the matrix-valued lift of the scalar oscillator of Section 7 (Remark 7.1). Let $Q_{t-1}, Q_t \in \mathbb{R}^{n \times d}$

be the previous two states. The forcing term is derived from the current transformer context $H_t = \text{TransformerBlock}(X_t)$,

$$B_t = \text{Linear}(H_t) \in \mathbb{R}^{n \times d}, \quad (22)$$

and the vibrational update is

$$Q_{t+1} = 2Q_t - Q_{t-1} - \Delta t^2 Q_t L_f^\top - \Gamma \odot (Q_t - Q_{t-1}) + \Delta t^2 B_t, \quad (23)$$

where $\Delta t > 0$ is a learnable time-step (scalar or diagonal), $\Gamma \in \mathbb{R}^{n \times d}$ is a learnable damping tensor initialised to a small positive constant, $Q_t L_f^\top$ applies the Laplacian to each token’s feature vector (coupling feature coordinates through the graph), and $\odot$ is the elementwise product. This is exactly the discretised damped vibration $M\ddot{x} + C\dot{x} + \nabla J = 0$ of (16) with stiffness L_f and forcing B_t , applied to all n tokens simultaneously: L_f penalises feature-direction differences across tokens according to edge weights, and Γ sets whether each mode is underdamped (oscillatory) or overdamped (diffusive).

Remark 8.1 (Stability). The explicit step (23) is stable when $\Delta t^2 \lambda_{\max}(L_f) \leq 2$. In practice we clip the learnable Δt by $\Delta t \leq \sqrt{2/\lambda_{\max}(L_f)}$ at each forward pass, or use a learnable implicit scheme analogous to the mass-aware resolvent $S_{\sigma, M}$ of Section 5, which is unconditionally stable.

8.5 Spectral formulation

Let $L_f = U\Lambda U^\top$ be the eigendecomposition, U orthogonal and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$, $\lambda_k \geq 0$. Define modal coordinates $\hat{Q}_t = Q_t U \in \mathbb{R}^{n \times d}$, so $\hat{Q}_{t,k}$ projects all token features onto the k -th eigenmode. In modal coordinates, (23) decouples mode-by-mode:

$$\hat{Q}_{t+1,k} = 2\hat{Q}_{t,k} - \hat{Q}_{t-1,k} - \Delta t^2 \lambda_k \hat{Q}_{t,k} - \Gamma_k(\hat{Q}_{t,k} - \hat{Q}_{t-1,k}) + \Delta t^2 \hat{B}_{t,k}, \quad (24)$$

a set of d independent damped harmonic oscillators, one per mode, with natural frequency $\omega_k = \sqrt{\lambda_k}$. This is exactly the modal analysis of Section 7, now indexed by the eigenmodes of L_f rather than of a loss Hessian, and it is Rayleigh’s classical modal decomposition [1]. Low-eigenvalue modes ($\lambda_k \approx 0$) are slowly varying global features; high-eigenvalue modes are rapidly varying local features. Learning adjusts Δt , Γ_k , and $\hat{B}_{t,k}$ to distribute representational capacity across the modal spectrum.

9 Signed Density Matrix State

9.1 Motivation

The vibrational state Q_t encodes feature-space dynamics but treats all modal occupancy as positive. In reasoning tasks it is natural to maintain both *supporting* and *contradicting* evidence for a hypothesis simultaneously, combining them through interference rather than simple accumulation. We formalise this with a signed density matrix, which generalises the positive-only Laplacian state of Part I.

9.2 Definition

Definition 9.1 (Signed density matrix state). The *signed density matrix state* at step t is a pair $(\varrho_t^+, \varrho_t^-)$ with $\varrho_t^+, \varrho_t^- \in \mathbb{R}^{m \times m}$, $m \leq d$ a chosen modal rank, both symmetric positive semidefinite, and the signed state

$$\varrho_t = \varrho_t^+ - \varrho_t^-. \quad (25)$$

Both components live in the Laplacian eigenbasis, i.e. the subspace spanned by the m lowest-energy eigenvectors $U_m = [u_1 | \dots | u_m]$ of L_f .

The diagonal entries $\varrho_{t,kk}^\pm$ are the positive/negative occupancies of mode k , while off-diagonals capture multi-mode coherences.

9.3 Update equations

Given the transformer context H_t , the projected context matrix $C_t = U_m^\top H_t^\top H_t U_m \in \mathbb{R}^{m \times m}$ is automatically PSD. Split it through learned affine maps,

$$C_t^+ = W_+^\top C_t W_+, \quad C_t^- = W_-^\top C_t W_-, \quad W_+, W_- \in \mathbb{R}^{m \times m}, \quad (26)$$

and update

$$\varrho_{t+1}^+ = (1 - \alpha)\varrho_t^+ + \alpha \text{softplus}(C_t^+) + \beta \varrho_t^+ \odot P_f, \quad (27)$$

$$\varrho_{t+1}^- = (1 - \alpha)\varrho_t^- + \alpha \text{softplus}(C_t^-) + \beta \varrho_t^- \odot P_f, \quad (28)$$

where $\alpha, \beta \in (0, 1)$ are learnable, $P_f = D_f^{-1}W_f$ is the graph Markov matrix of (8) (diffusion on feature modes), and $\text{softplus}(Z) = \log(1 + \exp(Z))$ applied entrywise (then symmetrised) preserves PSD structure. The diffusion term ties the density-matrix dynamics to the same feature-graph pathways P_f that govern the preconditioner and wave update.

9.4 Observables

From the signed density matrix we extract scalar observables used as diagnostics and as inputs to the prediction head:

$$E_t^\pm = \text{Tr}(\Lambda_m \varrho_t^\pm) \quad (\text{modal energy of positive/negative components}), \quad (29)$$

$$C_t^{\text{coh}} = \text{Tr}((\varrho_t^+ - \varrho_t^-)^2) \quad (\text{total coherence, squared Frobenius}), \quad (30)$$

$$\text{occ}_{t,k} = \varrho_{t,kk}^+ - \varrho_{t,kk}^- \quad (\text{signed modal occupancy per mode } k). \quad (31)$$

Inspecting the modal energy spectrum during training reveals which frequency bands the network uses most — the learning-time analogue of the taumode spectral coordinates of Section 6.

9.5 Reprojection into the VDT recurrence

The vibrational state is recovered from the density matrix by mixing the transformer context with the signed modal structure,

$$Q_t = H_t U_m \varrho_t U_m^\top + H_t, \quad (32)$$

before feeding back into (20). This reprojection is the VDT analogue of LDT’s lattice projection $\Pi_{\mathcal{L}}$: it constrains Q_t to respect the spectral geometry of L_f .

10 Training Objectives

The full VDT training objective is

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \mu_1 \mathcal{L}_{\text{freq}} + \mu_2 \mathcal{L}_{\text{depth}}, \quad (33)$$

where:

- $\mathcal{L}_{\text{task}}$ is the task-specific cross-entropy or MSE loss. Following LDT [15], we supervise every intermediate step $t = 1, \dots, K$ with the ground-truth label and average. This *depth supervision* forces the vibrational dynamics to represent useful partial conclusions at every recurrent depth.

- $\mathcal{L}_{\text{freq}}$ is the taumode frequency MSE of Section 6, lifted to the recurrent trajectory:

$$\mathcal{L}_{\text{freq}} = \frac{1}{K} \sum_{t=1}^K (\hat{\rho}(Q_t) - \lambda_{\star}^{\tau})^2, \quad (34)$$

biasing the vibrational trajectory toward the in-distribution spectral band of the ArrowSpace corpus.

- $\mathcal{L}_{\text{depth}}$ penalises excessive signed occupancy to prevent the density matrix from collapsing to a trivial state:

$$\mathcal{L}_{\text{depth}} = \frac{1}{K} \sum_{t=1}^K \|\varrho_t\|_F^2. \quad (35)$$

The objective thus inherits, term by term, the data-fidelity, spectral, and stability ingredients of the Part I cost functional (15), now expressed over a recurrent forward-only trajectory rather than a single least-squares solve.

11 Experiments

This section presents the planned experimental protocol; results will be reported in a subsequent revision. The protocol is designed for exact comparability with Davis et al. (2026) [15].

11.1 Experimental protocol following LDT

We adopt the three task families of Davis et al.:

1. **Propositional satisfiability (SAT):** random 3-SAT instances of increasing clause-to-variable ratio; binary SAT/UNSAT label. Train up to 20 variables, test up to 40.
2. **Syllogistic deduction:** multi-step categorical syllogisms (All A are B ; All B are C ; therefore All A are C) with up to 8 steps, tested on chain lengths unseen in training.
3. **Multi-step numeric reasoning:** integer arithmetic chains (e.g. iterative modular addition) of variable depth; accuracy vs. chain length.

For each task we run four architectures under identical training budgets (parameter count, compute steps, random seed sets):

Baseline	Description
Transformer	Vanilla recurrent transformer (shared weights, K steps)
LDT	Lattice Deduction Transformer [15]
VDT (ours)	This paper: wave update + density matrix
VDT-ablation	VDT without density matrix (plain Φ_L only)

11.2 Metrics

For each task and architecture we report: accuracy vs. recurrent depth $K \in \{2, 4, 8, 16\}$; accuracy vs. problem complexity (clause ratio / chain length / number of steps), including out-of-distribution lengths; modal energy spectra E_t^+, E_t^- as a function of depth (VDT only); and signed modal occupancy profiles $\{\text{occ}_{t,k}\}_{k=1}^m$ at convergence.

11.3 Hypotheses

From the theoretical properties of Φ_L and the signed density matrix we hypothesise: (i) VDT matches or exceeds LDT on syllogistic deduction, via the correspondence between modal low-energy states and globally consistent conclusions; (ii) VDT generalises to longer chains better than a plain recurrent transformer, because the spectral constraint prevents representation collapse at large K ; (iii) the signed density matrix helps on SAT (where contradictions matter) relative to the ablation; and (iv) modal energy spectra are interpretable signatures — SAT instances populate high- λ modes while definite syllogisms concentrate weight on low- λ modes.

12 Discussion

The VDT uses the feature Laplacian as both *geometry* (the spectral lattice of Part I) and *dynamics* (the constrained wave update of Part II). Replacing LDT’s logical lattice with a physical wave equation has three practical consequences.

Interpretability. Modal energies and occupancies are measurable during inference. For a given input one can inspect which frequency bands are activated and whether the signed density matrix has resolved (positive $\gg$ negative) or remains ambiguous (balanced) — a direct analogue of inspecting which atoms in a lattice are activated in LDT, and a recurrent counterpart to the taumode diagnostics of Part I.

Generalisation by spectral regularity. The Laplacian constraint keeps high-depth representations spectrally smooth; they cannot degenerate into arbitrary high-frequency noise. This is the recurrent analogue of the convergence-stabilising role played by the mass-aware preconditioner $S_{\sigma,M}$ in Proposition 5.1, and it mirrors lattice projection’s role in keeping LDT states within logically consistent partial derivations.

Connections to quantum machine learning. The signed density matrix $\varrho_t = \varrho_t^+ - \varrho_t^-$ is formally similar to a difference of quantum states, though we make no quantum claims. The analogy suggests techniques from open quantum systems (Lindblad operators, decoherence) as alternative formulations of the density-matrix update (27)–(28).

Limitations and future work. The explicit wave step (23) imposes a stability constraint on Δt (mitigated by the implicit resolvent of Part I). The density-matrix update doubles per-step state variables. The feature graph G_f must be built or updated at training time; for large d this needs efficient sparse Laplacian solvers, for which the ArrowSpace machinery is directly applicable. The taumode regulariser of Section 6 remains speculative, and full experimental results are pending. A formal stability characterisation of taumode coordinates across corpora, and the interaction between $\mathcal{L}_{\text{freq}}$ and tau-modulated retrieval, are natural next steps.

13 Conclusion

We have presented a single, self-contained account that runs from the mechanics of least-squares to a recurrent reasoning architecture. The foundations establish that feature space, equipped with a Laplacian stiffness L_f and mass M built from $A^\top$, is a Rayleigh mass-spring system whose generalised Rayleigh quotient governs preconditioning (with provable linear convergence), spectral regularisation, and the damped-oscillator behaviour of gradient descent. Lifting this mechanism from a single parameter vector to the hidden states of a recurrent transformer yields the Vibrational Deduction Transformer, in which a second-order discrete wave equation and a signed density

matrix in the Laplacian eigenbasis constrain every forward step to the spectral geometry of L_f — the structural analogue of lattice projection in the Lattice Deduction Transformer. The result is an architecture with an explicit mechanical pedigree, interpretable modal diagnostics, and a benchmark protocol for direct comparison against lattice-projection baselines; experimental validation is in progress.

Acknowledgements

This work made use of LLM tools to assist with drafting and editing; all technical claims remain the author’s responsibility. Thanks to [Enverge Lab](#) for compute access and to GitHub Sponsors for continued support; sponsor the project at github.com/sponsors/tuned-org-uk.

References

- [1] Lord Rayleigh. *The Theory of Sound, Volume I*, 2nd edition. Macmillan, 1894.
- [2] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Proc. NeurIPS*, 2022.
- [3] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. [arXiv:2408.03314](#), 2024.
- [4] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg. Structured denoising diffusion models in discrete state spaces. In *Proc. NeurIPS*, 2021.
- [5] G. H. Golub and C. F. Van Loan. *Matrix Computations*, 4th edition. Johns Hopkins University Press, 2013.
- [6] R. Merris. Laplacian matrices of graphs: a survey. *Linear Algebra and its Applications*, 197–198:143–176, 1994.
- [7] D. A. Spielman. Algorithms, graph theory, and linear equations in Laplacian matrices. In *Proc. ICM*, 2010.
- [8] X. Zhu, Z. Ghahramani, and J. Lafferty. Semi-supervised learning using Gaussian fields and harmonic functions. In *Proc. ICML*, pages 912–919, 2003.
- [9] M. Belkin, I. Matveeva, and P. Niyogi. Regularization and semi-supervised learning on large graphs. In *Proc. COLT*, pages 624–638, 2004.
- [10] A. J. Smola and R. Kondor. Kernels and regularization on graphs. In *Learning Theory and Kernel Machines*, pages 144–158. Springer, 2003.
- [11] X. Dong, D. Thanou, M. Rabbat, and P. Frossard. Learning Laplacian matrix in smooth graph signal representations. *IEEE Trans. Signal Processing*, 64(23):6160–6173, 2016.
- [12] S. Osher, W. Shi, and W. Zhu. Laplacian smoothing gradient descent. *Research in the Mathematical Sciences*, 9(3):Paper No. 39, 2022.
- [13] L. Moriondo. ArrowSpace: Spectral indexing for vector search. *Journal of Open Source Software*, 10(90):9002, 2025.

- [14] L. Moriondo. Graph Wiring and ArrowSpace: Spectral pathways for structure-aware learning. Technical report, 2026.
- [15] L. Davis, M. Jones, S. Kumar, and P. Williams. Lattice Deduction Transformers. arXiv:2605.08605v1, 2026.
- [16] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser. Universal Transformers. In *Proc. ICLR*, 2019.
- [17] OpenAI. OpenAI o1 System Card. Technical report, OpenAI, September 2024. <https://openai.com/index/openai-o1-system-card/>
- [18] OpenAI. OpenAI o3 and o4-mini System Card. Technical report, OpenAI, April 2025. <https://openai.com/index/o3-and-o4-mini-system-card/>

A Proof of Proposition 5.1

This appendix expands the proof of Proposition 5.1, the linear-convergence guarantee for mass-aware Laplacian-preconditioned gradient descent on $J(x) = \frac{1}{2m} \|Ax - b\|_2^2$, with $H = \frac{1}{m} A^\top A \succ 0$, $L_f \succeq 0$ symmetric, $M \succ 0$ diagonal, and $P_{\sigma,M} = (M + \sigma L_f)^{-1} M$, $\sigma > 0$.

A.1 Step 1: $P_{\sigma,M}$ is symmetric positive definite

Write $A_\sigma = M + \sigma L_f$. Since $M \succ 0$ diagonal and $L_f \succeq 0$, $A_\sigma \succ 0$ for every $\sigma > 0$, so A_σ^{-1} exists. Factor through the symmetric square root of M ,

$$A_\sigma = M^{1/2} (I + \sigma M^{-1/2} L_f M^{-1/2}) M^{1/2} =: M^{1/2} \tilde{A}_\sigma M^{1/2},$$

with $\tilde{A}_\sigma = I + \sigma \tilde{L}_f$ and $\tilde{L}_f = M^{-1/2} L_f M^{-1/2} \succeq 0$. Then $P_{\sigma,M} = A_\sigma^{-1} M = M^{-1/2} \tilde{A}_\sigma^{-1} M^{1/2}$ is congruent via $M^{1/2}$ to the SPD matrix $\tilde{A}_\sigma^{-1} = (I + \sigma \tilde{L}_f)^{-1}$, confirming $P_{\sigma,M} \succ 0$ and the existence of $P_{\sigma,M}^{1/2}$.

A.2 Step 2: positive spectrum of the preconditioned Hessian

The preconditioned Hessian $\mathcal{H}_{\sigma,M} = P_{\sigma,M} H$ is similar to a symmetric matrix:

$$P_{\sigma,M}^{-1/2} \mathcal{H}_{\sigma,M} P_{\sigma,M}^{1/2} = P_{\sigma,M}^{1/2} H P_{\sigma,M}^{1/2} =: \mathcal{S}_{\sigma,M},$$

which is SPD because $H \succ 0$ and $P_{\sigma,M}^{1/2} \succ 0$. Hence $\mathcal{H}_{\sigma,M}$ and $\mathcal{S}_{\sigma,M}$ share the same real, positive spectrum, and we set $\mu_{\sigma,M} = \lambda_{\min}(\mathcal{S}_{\sigma,M}) > 0$, $L_{\sigma,M} = \lambda_{\max}(\mathcal{S}_{\sigma,M}) < \infty$.

A.3 Step 3: contraction in the weighted norm

The error $e_t = x_t - x^*$ obeys $e_{t+1} = (I - \eta \mathcal{H}_{\sigma,M}) e_t$. Because $\mathcal{H}_{\sigma,M}$ is not symmetric in general, we measure the error in the $P_{\sigma,M}^{-1}$ -weighted norm $\|w\|_{P^{-1}}^2 = w^\top P_{\sigma,M}^{-1} w$. Under $\tilde{e}_t = P_{\sigma,M}^{-1/2} e_t$,

$$\tilde{e}_{t+1} = (I - \eta \mathcal{S}_{\sigma,M}) \tilde{e}_t,$$

with $I - \eta \mathcal{S}_{\sigma,M}$ symmetric and eigenvalues $1 - \eta \lambda$ for $\lambda \in [\mu_{\sigma,M}, L_{\sigma,M}]$. For $0 < \eta < 2/L_{\sigma,M}$ every such eigenvalue lies in $(-1, 1)$, so

$$\|I - \eta \mathcal{S}_{\sigma,M}\|_2 = \max\{|1 - \eta \mu_{\sigma,M}|, |1 - \eta L_{\sigma,M}|\} \leq 1 - \eta \mu_{\sigma,M} < 1.$$

Since $\|e_t\|_{P^{-1}} = \|\tilde{e}_t\|_2$, $\|x_{t+1} - x^*\|_{P^{-1}} \leq (1 - \eta \mu_{\sigma,M}) \|x_t - x^*\|_{P^{-1}}$.

A.4 Step 4: equivalence of norms

Because $P_{\sigma,M} \succ 0$,

$$\lambda_{\max}(P_{\sigma,M})^{-1/2} \|w\|_2 \leq \|w\|_{P^{-1}} \leq \lambda_{\min}(P_{\sigma,M})^{-1/2} \|w\|_2,$$

so iterating the contraction gives linear convergence in $\|\cdot\|_2$,

$$\|x_t - x^*\|_2 \leq \sqrt{\kappa(P_{\sigma,M})} (1 - \eta \mu_{\sigma,M})^t \|x_0 - x^*\|_2,$$

with $\kappa(P_{\sigma,M}) = \lambda_{\max}(P_{\sigma,M})/\lambda_{\min}(P_{\sigma,M})$. This completes the proof. $\square$

B PyTorch Implementation of the VDT Core

B.1 Feature graph and Laplacian

```
1 import torch
2 import torch.nn.functional as F
3
4 def build_feature_laplacian(X, k=5, sigma=None):
5     """
6     X : (n_tokens, d_features)
7     Returns L_f (d x d), W_f (d x d), eigenvalues, eigenvectors.
8     """
9     Xt = X.T # (d, n)
10    diff = Xt.unsqueeze(1) - Xt.unsqueeze(0) # (d, d, n)
11    dist2 = (diff ** 2).sum(-1) # (d, d)
12
13    if sigma is None:
14        sigma = dist2.mean().sqrt().item() + 1e-8
15
16    W = torch.exp(-dist2 / sigma**2)
17    W.fill_diagonal_(0.0)
18
19    # k-NN sparsification
20    topk_vals, _ = torch.topk(W, k=k, dim=-1)
21    threshold = topk_vals[:, -1:]
22    W = W * (W >= threshold).float()
23    W = (W + W.T) / 2 # symmetrise
24
25    d_vec = W.sum(dim=1) # degree vector
26    D = torch.diag(d_vec)
27    L = D - W
28
29    # Eigendecomposition
30    eigvals, eigvecs = torch.linalg.eigh(L) # sorted ascending
31    return L, W, eigvals, eigvecs
```

Listing 1: Build the feature-space Laplacian from a token embedding matrix.

B.2 Vibrational state block

```
1 import torch
2 import torch.nn as nn
3
4 class VibrationalStateBlock(nn.Module):
5     def __init__(self, d_feat, lambda_max, dt_init=0.1):
6         super().__init__()
7         self.d = d_feat
8         self.lambda_max = lambda_max
9         # Learnable time-step, clipped for stability
10        self.log_dt = nn.Parameter(torch.tensor(dt_init).log())
11        # Learnable per-feature damping
12        self.log_gamma = nn.Parameter(
13            torch.full((d_feat,), -2.0) # init gamma ~ 0.135
```

```

14         )
15         # Forcing projection
16         self.forcing = nn.Linear(d_feat, d_feat, bias=False)
17
18     def forward(self, Q_prev, Q_curr, H_t, L_f):
19         """
20         Q_prev, Q_curr : (n, d)   vibrational states at t-1 and t
21         H_t           : (n, d)   transformer context at t
22         L_f           : (d, d)   feature Laplacian
23         Returns Q_next : (n, d)
24         """
25         # Clip dt for CFL stability
26         dt_max = (2.0 / (self.lambda_max + 1e-8)) ** 0.5
27         dt = torch.exp(self.log_dt).clamp(max=dt_max)
28         gamma = torch.exp(self.log_gamma)           # (d,)
29         B_t = self.forcing(H_t)                     # (n, d)
30
31         # Wave update:  $Q_{t+1} = 2Q - Q_{prev} - dt^2 Q L^T$ 
32         #  $- gamma*(Q - Q_{prev}) + dt^2 B$ 
33         stiffness = dt**2 * (Q_curr @ L_f.T)         # (n, d)
34         damp = gamma.unsqueeze(0) * (Q_curr - Q_prev) # (n, d)
35         Q_next = (2 * Q_curr - Q_prev
36                 - stiffness
37                 - damp
38                 + dt**2 * B_t)
39         return Q_next

```

Listing 2: Vibrational state block implementing the discrete wave update.

B.3 Signed density matrix

```

1  class SignedDensityMatrix(nn.Module):
2      def __init__(self, m_modes):
3          """m_modes: number of low-energy modes to retain."""
4          super().__init__()
5          self.m = m_modes
6          self.W_plus = nn.Parameter(torch.eye(m_modes))
7          self.W_minus = nn.Parameter(torch.eye(m_modes) * 0.1)
8          self.alpha = nn.Parameter(torch.tensor(0.3))
9          self.beta = nn.Parameter(torch.tensor(0.1))
10
11     def forward(self, rho_plus, rho_minus, H_t, U_m, P_f_m):
12         """
13         rho_plus, rho_minus : (m, m)   current PSD states
14         H_t                 : (n, d)   transformer context
15         U_m                 : (d, m)   first m eigenvectors of L_f
16         P_f_m               : (m, m)   P_f restricted to m modes
17         Returns updated rho_plus, rho_minus : (m, m)
18         """
19         alpha = self.alpha.sigmoid()
20         beta = self.beta.sigmoid()
21
22         # Context projected to modal subspace
23         C = U_m.T @ H_t.T @ H_t @ U_m           # (m, m)

```

```

24
25     # Positive / negative context projections
26     C_plus = self.W_plus.T @ C @ self.W_plus
27     C_minus = self.W_minus.T @ C @ self.W_minus
28
29     # Ensure PSD via softplus (element-wise, then symmetrise)
30     def make_psd(Z):
31         Z = F.softplus(Z)
32         return (Z + Z.T) / 2
33
34     rho_plus_new = ((1-alpha)*rho_plus
35                     + alpha*make_psd(C_plus)
36                     + beta*(rho_plus * P_f_m))
37     rho_minus_new = ((1-alpha)*rho_minus
38                     + alpha*make_psd(C_minus)
39                     + beta*(rho_minus * P_f_m))
40     return rho_plus_new, rho_minus_new
41
42     def to_vibrational_state(self, rho_plus, rho_minus,
43                             H_t, U_m):
44         """Reprojection: mix signed density matrix into transformer
45         context."""
46         rho = rho_plus - rho_minus # (m, m)
47         modal_correction = H_t @ U_m @ rho @ U_m.T # (n, d)
48         return H_t + modal_correction

```

Listing 3: Signed density-matrix state in the Laplacian eigenbasis.

B.4 Full VDT forward pass

```

1  class VDT(nn.Module):
2      def __init__(self, d, m_modes, K, n_heads=4, lambda_max=1.0):
3          super().__init__()
4          self.K = K
5          self.transformer = nn.TransformerEncoderLayer(
6              d_model=d, nhead=n_heads, batch_first=True
7          )
8          self.vib_block = VibrationalStateBlock(d, lambda_max)
9          self.density = SignedDensityMatrix(m_modes)
10         self.m = m_modes
11
12     def forward(self, X0, L_f, eigvecs, P_f):
13         """
14         X0      : (batch, n, d)
15         L_f     : (d, d)
16         eigvecs: (d, d) columns are eigenvectors of L_f
17         P_f     : (d, d) Markov matrix on feature graph
18         """
19         B, n, d = X0.shape
20         U_m = eigvecs[:, :self.m] # (d, m) low-energy modes
21         P_m = U_m.T @ P_f @ U_m   # (m, m)
22
23         Q_prev = X0.clone()
24         Q_curr = X0.clone()

```

```

25     rho_p = torch.zeros(self.m, self.m)
26     rho_m = torch.zeros(self.m, self.m)
27
28     logits_all = []
29     for t in range(self.K):
30         # 1. Transformer mixing
31         H_t = self.transformer(Q_curr)          # (B, n, d)
32
33         # 2. Vibrational update (batch dimension: loop or vmap)
34         Q_next_list = []
35         for b in range(B):
36             Qn = self.vib_block(Q_prev[b], Q_curr[b],
37                                H_t[b], L_f)
38             Q_next_list.append(Qn)
39         Q_next = torch.stack(Q_next_list)      # (B, n, d)
40
41         # 3. Density matrix update (averaged over batch)
42         H_avg = H_t.mean(0)                  # (n, d)
43         rho_p, rho_m = self.density(
44             rho_p, rho_m, H_avg, U_m, P_m
45         )
46
47         # 4. Reprojection
48         X_repr_list = []
49         for b in range(B):
50             Xr = self.density.to_vibrational_state(
51                 rho_p, rho_m, Q_next[b], U_m
52             )
53             X_repr_list.append(Xr)
54         X_repr = torch.stack(X_repr_list)     # (B, n, d)
55
56         # Collect per-depth logits for depth supervision
57         logits_all.append(X_repr[:, 0, :])    # CLS token
58
59         Q_prev = Q_curr
60         Q_curr = X_repr
61
62     return Q_curr, logits_all, (rho_p, rho_m)

```

Listing 4: Sketch of the full VDT recurrent forward pass.

C LDT-Mirrored Benchmark Tasks

This appendix specifies the exact task generation procedures for the three benchmark families, designed to mirror Davis et al. (2026) [15] as closely as possible.

C.1 Task 1: Propositional SAT

Random 3-SAT instances generated using Dimitriou–Spirakis clause sampling. Training: clause-to-variable ratios $\alpha \in [3.0, 4.0]$, 10–20 variables. Test (in-distribution): same ratios and sizes. Test (OOD): 20–40 variables. Tokenisation: each clause is a token; variables are embedded as learnable vectors.

C.2 Task 2: Syllogistic deduction

Chains $A_1 \subset A_2 \subset \dots \subset A_K$ encoded as “All A_i are A_{i+1} ” statements. Label: whether $A_1 \subset A_K$ (always true). Training chain length $K \in \{2, 3, 4\}$; test (OOD) $K \in \{5, 6, 7, 8\}$.

C.3 Task 3: Multi-step numeric reasoning

Iterative modular addition chains $x_0 \xrightarrow{+a_1} x_1 \xrightarrow{+a_2} \dots \xrightarrow{+a_K} x_K \pmod{p}$. Given $(x_0, a_1, \dots, a_K)$, predict x_K . Training: $K \leq 4$, $p = 97$; test (OOD): $K \in \{5, \dots, 10\}$.